

Meeting Assistant Technical Implementation

Document 2 of 3 · Engineering reference for the current MVP · [Overview](#) · [Roadmap](#) · [Changelog](#)

This document is the engineering reference for the running system: how data flows from input to Jira, how each component is built, the API surface, the user interface, error handling, and the test suite. For the motivation and product vision see [Document 1](#); for what comes next see [Document 3](#).

Contents

- | | |
|--|---|
| 1. System Goal | 9. API Endpoints |
| 2. MVP vs. Full Vision (YAGNI) | 10. Synthetic Data |
| 3. End-to-End Data Flow | 11. Environment Configuration |
| 4. Voice Transcription | 12. Project Structure |
| 5. SQLite Meeting Store | 13. Running the System |
| 6. ChromaDB Vector Store | 14. Tests |
| 7. Pydantic-AI Agents | 15. Error Handling |
| 8. User Interface | |

1. System Goal

Convert a Persian meeting transcript into operational output: a **summary**, **tasks**, a **RAG question-and-answer layer**, and **Jira issues** — through a pipeline that is fast enough to demonstrate live.

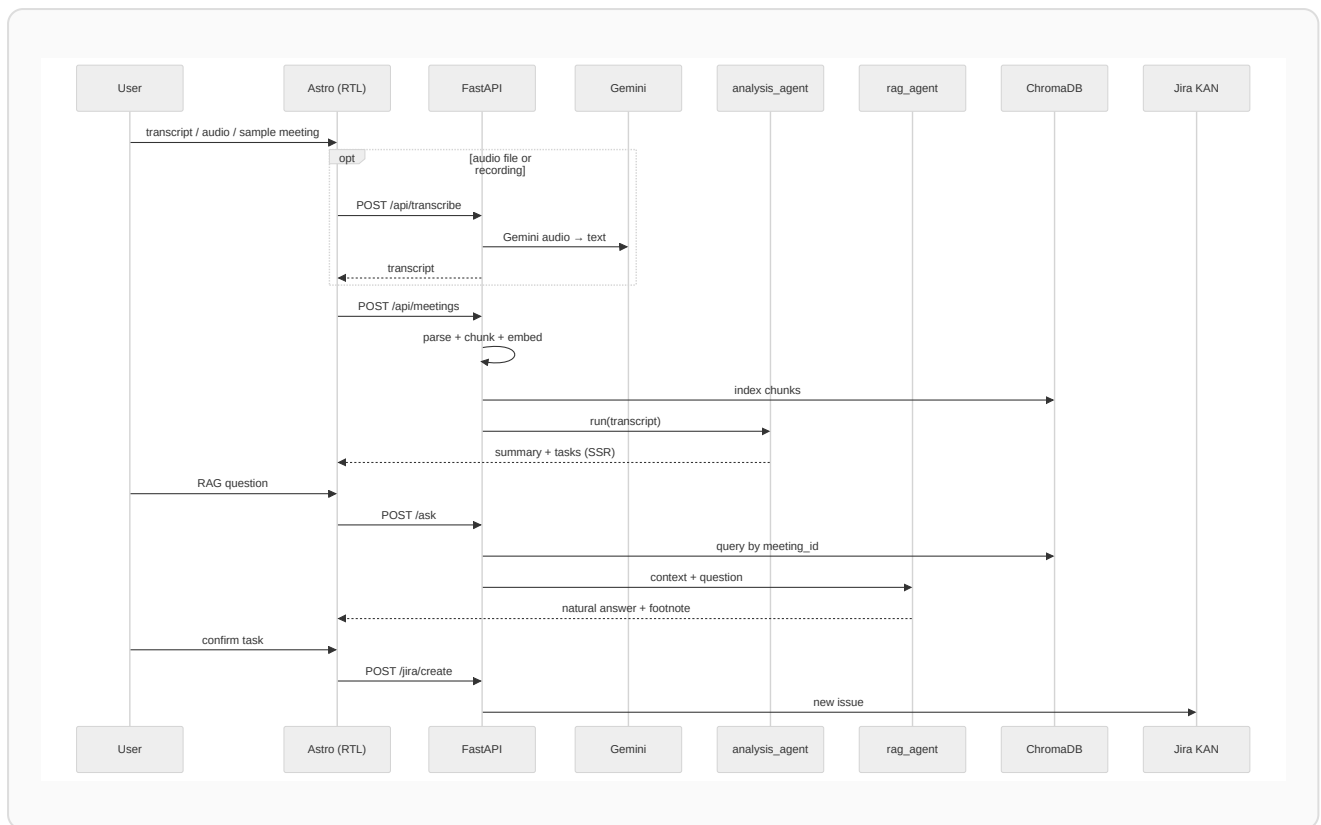
2. MVP vs. Full Vision (YAGNI)

The original concept document described a maximal system. The MVP keeps only what is needed to prove the value, and records explicitly what was deferred — a disciplined "you

aren't gonna need it" stance.

Capability	Full vision (concept doc)	Current MVP
Input	Live recording + streaming STT	Transcript (paste + text file) and audio (upload/record → Gemini)
Analysis	GPT-4 / mixed models	Gemini Flash + Pydantic AI
Search / vector DB	ElasticSearch	ChromaDB embedded (data/chroma/)
Task tracking	Jira + Trello + Asana + ...	Jira only (KAN) — issues in English
RAG	Whole-organization archive	Single meeting · multi-meeting planned (Sprint 2)
Calendar / Teams	Full integration	Deferred (later sprint)
Dashboard / notifications	Weekly reports	Cross-meeting task view today · richer reports deferred

3. End-to-End Data Flow



4. Voice Transcription (Gemini)

Audio input is a **separate step before ingest**, so pasting text and loading sample meetings remain unchanged. It reuses the same `GOOGLE_API_KEY` and `GEMINI_MODEL` — no separate speech-to-text service.

Step	Detail
UI	Upload <code>mp3</code> , <code>wav</code> , <code>m4a</code> , <code>ogg</code> , <code>webm</code> , <code>flac</code> or use the Record button (MediaRecorder).
API	<code>POST /api/transcribe</code> — multipart <code>file</code> → Gemini <code>generateContent</code> with inline audio.
Output	Persian transcript formatted as <code>[MM:SS] name: text</code> , editable before analysis.
Limit	Up to 20 MB (<code>MAX_AUDIO_BYTES</code>).
After transcription	The normal <code>POST /api/meetings</code> path — analysis, Chroma, RAG, Jira.

Difference from professional STT: diarization and live streaming are not part of the MVP; for demos and pre-recorded audio, single-shot Gemini transcription is sufficient.

5. SQLite — Meeting Store

Item	Value
File	<code>data/meetings.db</code>
Table	<code>meetings</code> — transcript, summary, key_points, decisions, tasks (JSON)
Migration	Legacy <code>data/meetings/*.json</code> files are imported once on first run.

SQLite gives the MVP durable, zero-operations persistence. The schema is deliberately simple so the Sprint 5 migration to PostgreSQL (via SQLAlchemy + Alembic) is straightforward.

6. ChromaDB — Vector Store

Why Chroma? An earlier version used a hand-written JSON store with a Python cosine loop — slow for demos and hard to maintain. ChromaDB is embedded: no separate server, persisted in `data/chroma/`, with fast HNSW search.

Item	Value
Library	<code>chromadb</code>
Collection	<code>meeting_chunks</code>
Filter	<code>metadata.meeting_id</code>
Embedding	<code>gemini-embedding-001</code> (from Google API; Chroma only stores/searches)
Distance	cosine (HNSW)

7. Pydantic-AI Agents

7.1 analysis_agent

```
from pydantic import BaseModel
from pydantic_ai import Agent

class MeetingAnalysis(BaseModel):
    title: str
    summary: str
    key_points: list[str]
    decisions: list[str]
    tasks: list[MeetingTask]

analysis_agent = Agent(
    'google:gemini-3.5-flash',
    output_type=MeetingAnalysis,
    system_prompt='Analyze a Persian meeting transcript. Use only the text.',
)
```

7.2 RAG — smart behavior

- **Greetings / hello:** no transcript search — a short reply that invites a real question about the meeting.
- **Meeting question:** Chroma top-4 → internal context for the LLM (the raw transcript is never dumped into the UI).
- **Answer:** natural Persian with attribution ("Ali said...") and a short footnote when useful.

```
# small-talk detection
is_small_talk("hello") -> chitchat_agent, sources=[]

# real question
chunks = chroma.query(meeting_id, question)
rag_agent.run(context + question) -> RagAnswer
```

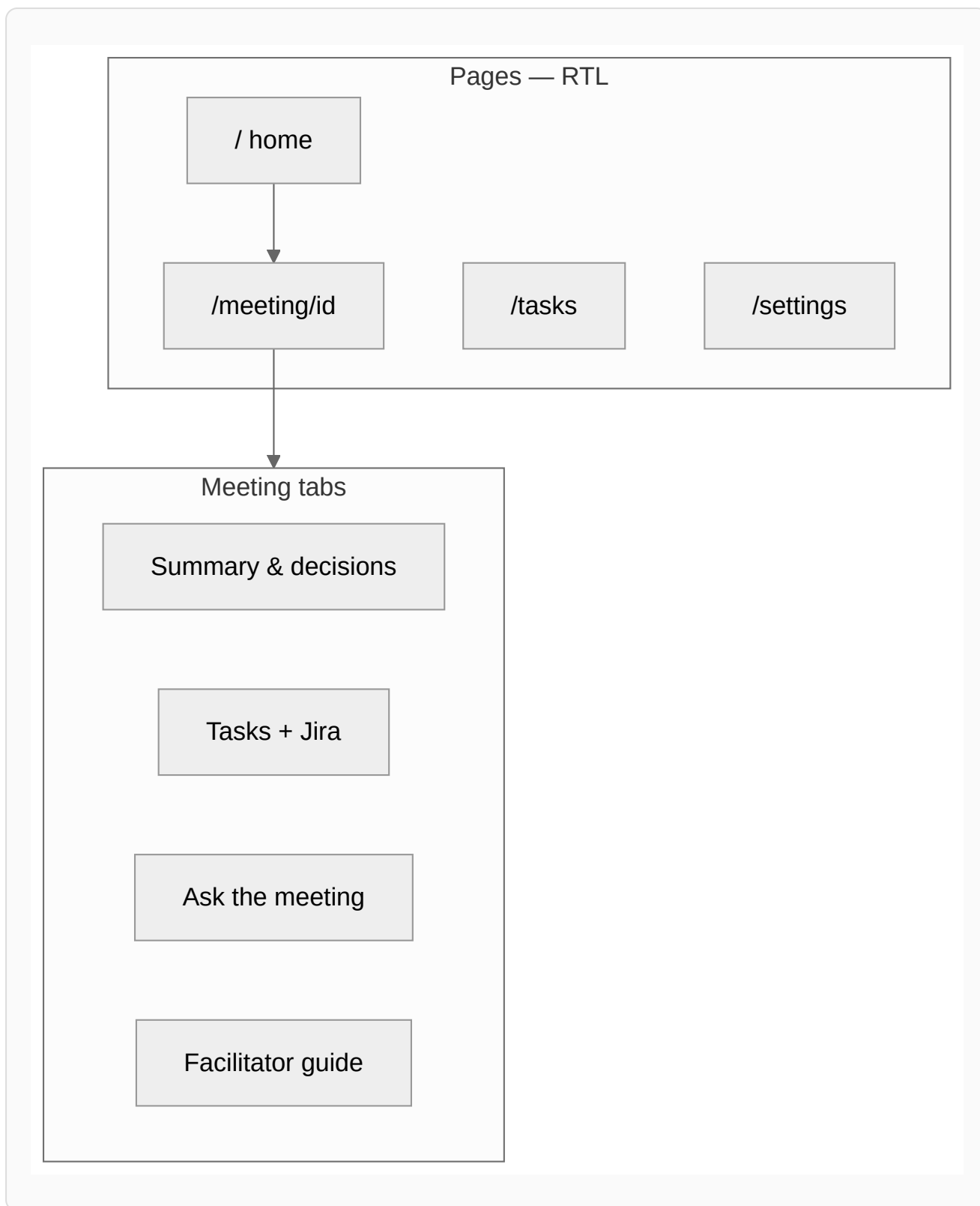
7.3 facilitation_agent

After ingest, the organizer can see suggestions for improving the next meeting — a coaching tone, not a scolding one.

- **Input:** transcript, `MeetingAnalysis` , speaker participation stats, meeting type.
- **Output:** `FacilitationReport` — what went well, improvements, next-meeting agenda, timebox suggestion, coaching summary, and an optional 1–5 facilitator score.
- **API:** `GET /api/meetings/{id}/facilitation`
- **UI:** a "Facilitator Guide" tab on the meeting page (lazy-loaded).

Jira creation is a direct API route. Issues are written in **English** (`title_en` , `context_en`) for clean left-to-right display in Jira, while the UI stays Persian.

8. User Interface (Astro SSR)



RTL / LTR rules:

- `html lang="fa" dir="rtl"` — Persian text is right-aligned.
- Code, env vars, URLs, and API names are wrapped in `dir="ltr"`.
- Mermaid diagrams render inside an LTR wrapper.

The `/meeting/[id]` page fetches its data on the **Astro server** from FastAPI (not only client-side JS), which fixes the "stuck on loading" problem and improves first paint.

Home — meeting archive, search/filter, synthetic meetings, ingest form

Meeting · Summary — key points, decisions, participation stats

Meeting · Tasks — action items, preview and create Jira issues

Meeting · Ask — RAG with natural answers and sources

Tasks — cross-meeting dashboard with an "open only" filter

Settings — API status, speaker → Jira mapping, .env guide

Screenshots are produced by `./scripts/capture-ui-screenshots.sh` (Playwright) — backend + frontend must be running.

9. API Endpoints

Method	Path	Description
GET	<code>/api/health</code>	Service health — Google/Jira/DB status
GET	<code>/api/meetings</code>	List meetings
GET	<code>/api/meetings/{id}</code>	Get one meeting
DELETE	<code>/api/meetings/{id}</code>	Delete a meeting
GET	<code>/api/tasks</code>	Cross-meeting action items

GET	<code>/api/meetings/synthetic</code>	List bundled synthetic meetings
POST	<code>/api/meetings/synthetic/{file_id}</code>	Ingest a synthetic meeting
POST	<code>/api/transcribe</code>	Transcribe an audio file with Gemini
POST	<code>/api/meetings</code>	Ingest + <code>analysis_agent</code>
GET	<code>/api/meetings/{id}/export</code>	Export a meeting summary
GET	<code>/api/meetings/{id}/speakers</code>	Speaker participation stats
GET	<code>/api/meetings/{id}/facilitation</code>	Facilitator guide — <code>facilitation_agent</code>
POST	<code>/api/meetings/{id}/ask</code>	RAG — <code>rag_agent</code>
POST	<code>/api/meetings/{id}/jira/preview</code>	Preview Jira issues
POST	<code>/api/meetings/{id}/jira/create</code>	Create issues in Jira
GET / PUT / DELETE	<code>/api/settings/assignee-map</code>	Speaker → Jira account mapping

10. Synthetic Data

File	Topic
<code>meeting-01-scrum.txt</code>	Weekly stand-up
<code>meeting-02-planning.txt</code>	MVP planning
<code>meeting-03-review.txt</code>	Sprint review

Line format: `[14:02] Ali: spoken text .`

11. Environment Configuration (.env)

```
GOOGLE_API_KEY=...
EMBEDDING_MODEL=gemini-embedding-001
GEMINI_MODEL=google:gemini-3.5-flash
MAX_AUDIO_BYTES=20971520
JIRA_SITE_URL=https://your-site.atlassian.net
JIRA_PROJECT_KEY=KAN
JIRA_EMAIL=your@email.com
JIRA_API_TOKEN=...
PUBLIC_BACKEND_URL=http://127.0.0.1:8000
BACKEND_PORT=8000
```

12. Project Structure

```
project/
├── frontend/           # Astro SSR – Persian RTL UI
├── backend/
│   ├── main.py         # FastAPI app + routes
│   ├── agents/         # analysis_agent, rag_agent, facilitation
│   ├── services/       # sqlite, chroma, jira, embed, audio_transcribe,
│   │                   # gemini_errors, stats, ingest, facilitation
│   ├── models/         # Pydantic schemas
│   └── tests/          # unit + live tests
├── data/synthetic/     # sample meetings
├── data/meetings.db    # SQLite – summaries and tasks
├── data/chroma/        # ChromaDB persistence
├── docs/              # HTML docs + PDFs + diagrams + CHANGELOG
└── scripts/           # run-dev, run-backend, run-frontend, tests, export-docs
```

13. Running the System

The whole stack starts with a single script (frees ports, starts both servers, cleans up on exit):

```
chmod +x scripts/run-dev.sh
./scripts/run-dev.sh
# open http://localhost:4321
```

Or run the two services separately:

```
./scripts/run-backend.sh    # FastAPI on :8000
./scripts/run-frontend.sh   # Astro on :4321
```

14. Tests

Type	Command	Count
Unit (mocked)	<code>./scripts/run-tests.sh</code>	150 passing (13 skip without optional/live creds)
Live API & integration	<code>./scripts/run-live-tests.sh</code>	13 (real Gemini/Jira)
All	<code>./scripts/run-all-tests.sh</code>	unit + live

Unit tests mock the LLM and external services so they run fast and offline. The live suite exercises the real FastAPI routes against real Google/Jira credentials, with a single end-to-end Gemini flow per session to avoid event-loop issues with Pydantic AI.

15. Error Handling

Error	Behavior
Gemini timeout	One retry + a Persian message
Transcript with no speaker	Labelled <code>[Unknown]</code>
Jira 401	Guidance to check <code>.env</code>
Invalid / oversized audio	HTTP 400 (Persian) — format or <code>MAX_AUDIO_BYTES</code>
Gemini billing (403 dunning)	Friendly Persian billing message for transcription and analysis

RAG with no match	"Not found in this meeting"
-------------------	-----------------------------

A dedicated `gemini_errors` service maps provider exceptions (`ModelHTTPError` and `httpx errors`) to user-friendly Persian `HTTPException` s, so a billing hold or an invalid key never surfaces as a raw stack trace.

Next sprints: see [Document 3](#) — multi-meeting RAG + full-text archive (Sprint 2), Jira assignee mapping + OAuth (Sprint 3), auth/workspace + calendar metadata (Sprint 4), PostgreSQL + queue + deployment (Sprint 5), and multi-agent capabilities (Sprint 6+). Live streaming STT is intentionally *not* on the roadmap — Gemini transcription covers the MVP need.